

UNIT – 5 MLOps

ML Research vs Production – Creating experiments with Weights and Biases(W&B) – Logging training information to W&B – Experiment Tracking with W&B - Storing artefacts with W&B – Model Versioning with W&B – Data Versioning with W&B

ML Research vs Production

1. Goal and Focus

- **ML Research:**
 - Focuses on advancing theoretical understanding, discovering new algorithms, and improving existing models.
 - Main goal: Solve a problem with high accuracy using innovative approaches.
 - Usually involves experimentation, comparison of techniques, and validation against benchmarks.
 - **ML Production:**
 - Emphasizes scalability, reliability, and real-world deployment.
 - Main goal: Integrate the model into a system to deliver continuous, real-time predictions or services.
 - Focus is on operational efficiency, including inference speed, monitoring, and performance optimization.
-

2. Data Handling

- **ML Research:**
 - Often uses well-curated, clean, and labeled datasets (e.g., MNIST, CIFAR-10) for experimentation.
 - Data may not represent real-world complexities, such as missing or noisy data.
 - Small to moderate dataset sizes are typical for benchmarking algorithms.
 - **ML Production:**
 - Deals with real-world, large-scale, often noisy, incomplete, and unbalanced datasets.
 - Requires handling issues like missing data, incorrect labels, streaming data, and evolving data distributions.
 - Data pipeline management is crucial for continuous model updates and monitoring.
-

3. Model Complexity and Evaluation

- **ML Research:**
 - Focuses on building and tuning complex models like deep neural networks, transformers, etc., to push state-of-the-art performance.
 - Evaluation typically revolves around metrics like accuracy, precision, recall, F1-score, etc.
 - Models may not be optimized for speed or resource efficiency since the focus is on theoretical performance.
 - **ML Production:**
 - Models must balance between complexity and practicality (e.g., trade-offs between accuracy and speed).
 - Evaluation criteria expand to include latency, throughput, memory usage, and energy consumption.
 - Continuous model evaluation and monitoring for "model drift" or "data drift" are essential.
-

4. Model Deployment

- **ML Research:**
 - Deployment is often not a priority. The focus is more on publishing results and sharing code/models (e.g., on GitHub).
 - No requirement for real-time interaction or distributed inference.
 - **ML Production:**
 - Deployment is a critical aspect. The model is served via APIs, integrated with applications, or used in real-time environments.
 - Scaling, low-latency inference, A/B testing, and model versioning are essential considerations.
 - Distributed inference across cloud servers or edge devices is common in production systems.
-

5. Infrastructure and Tooling

- **ML Research:**
 - Limited use of large-scale infrastructure; often uses local machines, academic clusters, or limited cloud resources.
 - Tools used include research-oriented libraries like TensorFlow, PyTorch, Scikit-learn, and experiment management tools like Weights and Biases.
- **ML Production:**

- Requires robust infrastructure for continuous training, deployment, and monitoring.
 - Use of cloud platforms (e.g., AWS, GCP, Azure), containerization (Docker), orchestration (Kubernetes), and CI/CD pipelines for automating updates.
 - Monitoring tools like Prometheus, Grafana, and tools for version control, logging, and alerting are crucial.
-

6. Scalability and Performance

- **ML Research:**
 - Scalability concerns are minimal. The focus is on proving concepts, so models are often trained on small-scale datasets or environments.
 - Training times can be long, but efficiency is less of a priority compared to performance improvement.
 - **ML Production:**
 - Scalability is a top priority. Models must serve millions of requests or process large volumes of data efficiently.
 - Optimizing inference speed, reducing computational costs, and ensuring models work efficiently in distributed systems are key.
 - Techniques like model compression, quantization, and pruning are often used to reduce the model size and make it feasible for production environments.
-

7. Iterative Development

- **ML Research:**
 - The iteration focuses on model experimentation, often driven by improving a specific metric (e.g., accuracy).
 - Slow iterations: Model updates occur when significant improvements or findings are made.
 - Research can afford "failure" as part of the discovery process.
 - **ML Production:**
 - Iteration focuses on continuous integration and deployment (CI/CD) cycles, ensuring the model improves and adapts to changing data patterns.
 - Fast iterations: Models need to be retrained frequently (e.g., daily or weekly) to reflect updated data and ensure performance consistency.
 - Failure in production impacts users, so high reliability is critical.
-

8. Testing and Validation

- **ML Research:**
 - Validation typically involves comparing different algorithms on test datasets and reporting performance metrics.
 - Unit tests or integration tests are rarely employed, as the goal is to explore ideas, not necessarily to ship them.
 - **ML Production:**
 - Testing is rigorous, including unit tests, integration tests, and functional tests to ensure model reliability and accuracy in production environments.
 - Shadow testing, rollback strategies, and A/B testing are often used to validate the model before full deployment.
-

9. Security and Compliance

- **ML Research:**
 - Security and compliance are not usually a concern since the model is not deployed in sensitive or real-world environments.
 - Code sharing and open-source contributions are common, with less focus on proprietary code protection.
 - **ML Production:**
 - Models must adhere to security protocols, data privacy laws (GDPR, HIPAA), and compliance requirements.
 - Data security, model interpretability, and explainability are often required, especially in sensitive applications like healthcare or finance.
-

10. Collaboration

- **ML Research:**
 - Typically involves a small group of researchers or individuals working independently.
 - Collaboration is more academic in nature, with peer reviews and feedback from the research community.
- **ML Production:**
 - Requires collaboration across teams: data engineers, software developers, DevOps, security, and business stakeholders.
 - Interdisciplinary work is essential to ensure the model is aligned with business goals, security standards, and operational efficiency.

Creating experiments with Weights and Biases(W&B) – Logging training information to W&B – Experiment Tracking with W&B - Storing artefacts with W&B

1. Introduction to Weights and Biases (W&B)

Weights and Biases (W&B) is a popular tool for tracking and managing machine learning experiments. It provides an easy interface to:

- Track model performance.
- Log and visualize training metrics.
- Collaborate with teams.
- Store model artifacts like datasets, trained models, and more.
- Compare experiments easily for hyperparameter tuning and model optimization.

W&B integrates well with most machine learning frameworks like TensorFlow, PyTorch, Scikit-learn, etc.

2. Setting up Weights and Biases

a. Install W&B

You can install W&B using pip:

```
bash

pip install wandb
```

 Copy code

b. Login to W&B

After installing, login to W&B using the following command:

```
bash

wandb login
```

 Copy code

This will prompt you to enter an API key, which you can get from the W&B website after creating an account.

3. Creating Experiments with W&B

a. Initializing a W&B Run

The core concept in W&B is the **run**. A run refers to an experiment or a single execution of training a model. You initialize a run using:

```
python Copy code  
  
import wandb  
  
# Start a new run  
wandb.init(project="my-ml-project", entity="my-username")
```

- `project`: Name of the project where you want to log your experiments.
- `entity`: Your W&B username or team name.

b. Log Hyperparameters

You can log the hyperparameters used in an experiment for easy tracking:

```
python Copy code  
  
wandb.config.learning_rate = 0.001  
wandb.config.epochs = 10  
wandb.config.batch_size = 32
```

This helps track what parameters were used for each run and compare runs with different parameter values.

4. Logging Training Information to W&B

a. Log Training Metrics

You can log training metrics such as loss, accuracy, precision, etc., during or after each training epoch/iteration:

```
python Copy code  
  
for epoch in range(epochs):  
    # Training logic here...  
    loss = 0.5 # Example loss  
    accuracy = 0.85 # Example accuracy  
  
    # Log metrics  
    wandb.log({"epoch": epoch, "loss": loss, "accuracy": accuracy})
```

By logging metrics during training, W&B allows you to visualize the evolution of these metrics in real-time via its dashboard.

b. Log Custom Metrics

You can also log custom metrics that may be specific to your experiment:

```
python Copy code  
  
wandb.log({"custom_metric": value})
```

These can be domain-specific metrics, such as F1-score, AUC-ROC, or any user-defined metric.

c. Log Scalars, Histograms, and Images

- **Scalars:** Can be used to track metrics like loss and accuracy.
- **Histograms:** Can be used to log model weight distributions or gradient distributions.
- **Images:** Can log predictions, visualizations, or samples.

5. Experiment Tracking with W&B

a. Track Multiple Runs

Each run in W&B is automatically logged to the dashboard, allowing you to easily compare different experiments. You can create multiple runs, each with different hyperparameters or model architectures:

```
python Copy code  
  
for learning_rate in [0.001, 0.01, 0.1]:  
    # Initialize a new run for each experiment  
    run = wandb.init(project="my-ml-project", config={"learning_rate": learning_rate})  
  
    # Training Logic...  
  
    run.finish()
```

This will log different experiments in the same project, allowing you to compare performance visually.

b. Compare Runs

On the W&B dashboard, you can compare runs easily:

- View all logged metrics side by side.
- Plot them on the same graph for better comparisons.
- Use the **parallel coordinates plot** to visualize the impact of hyperparameters on performance.

c. Hyperparameter Sweeps

W&B supports hyperparameter sweeps, allowing automated search for the best hyperparameter configuration:

```
python Copy code  
  
sweep_config = {  
    'method': 'random',  
    'parameters': {  
        'learning_rate': {'values': [0.001, 0.01, 0.1]},  
        'batch_size': {'values': [16, 32, 64]},  
    }  
}  
  
sweep_id = wandb.sweep(sweep_config, project="my-ml-project")  
wandb.agent(sweep_id, function=train_function)
```

This will automatically run multiple experiments with different hyperparameter combinations.

6. Storing Artifacts with W&B

Artifacts in W&B are used to store any files generated during experiments, such as:

- Trained models.
- Datasets.
- Results like predictions.
- Visualizations.

a. Saving and Uploading Models

You can save trained models as artifacts to keep a history of them for later use:

```
python

# Save the model
torch.save(model.state_dict(), "model.pth")

# Log the model as an artifact
artifact = wandb.Artifact('model', type='model')
artifact.add_file('model.pth')
wandb.log_artifact(artifact)
```

b. Loading Artifacts

You can load an artifact from W&B in a subsequent run:

```
python

# Download and load a model artifact
artifact = wandb.use_artifact('model:latest')
artifact_dir = artifact.download()

# Load the model
model.load_state_dict(torch.load(f"{artifact_dir}/model.pth"))
```

 Copy code

This ensures that your models and data artifacts are well-organized and easy to access for future experiments or production use.

c. Dataset Artifacts

If you are working with large datasets, you can log them as artifacts:

python

 Copy code

```
artifact = wandb.Artifact('dataset', type='dataset')
artifact.add_file('data.csv')
wandb.log_artifact(artifact)
```

This helps with versioning datasets, especially in cases where you may update or modify your datasets over time.

a. Reports and Dashboards

You can create comprehensive reports on W&B that include:

- Visualizations of training metrics.
- Model performance comparisons.
- Detailed explanations or notes on experiments.

These reports are useful for documenting experiments and sharing insights with stakeholders.

8. Best Practices for Using W&B

- **Log Early and Often:** Log hyperparameters, training metrics, and artifacts throughout the experiment for easier tracking and comparison.
- **Use Sweeps for Optimization:** Automate the hyperparameter search using W&B sweeps.
- **Compare Experiments:** Regularly compare runs and tweak hyperparameters or models based on insights from W&B graphs.
- **Collaborate:** Share your W&B dashboards with team members for better collaboration and faster development cycles.
- **Version Control for Artifacts:** Keep track of model and dataset versions using W&B artifacts.

Model Versioning with W&B – Data Versioning with W&B

1. Model Versioning with W&B

a. Why Model Versioning is Important

In machine learning, model versioning helps track different versions of a model as it evolves. This is crucial for:

- **Reproducibility:** To ensure you can reproduce past experiments.
- **Model Comparison:** To compare older models with newer ones.

- **Deployment:** To roll back to previous versions if the new model underperforms.

W&B makes model versioning seamless by allowing you to track, store, and manage model versions as artifacts.

b. Steps to Implement Model Versioning with W&B

1. Logging a Model as an Artifact

You can log your trained model to W&B as an artifact, which helps to save a version of your model for future reference.

```
import torch
```

```
import wandb
```

```
# Initialize a W&B run
```

```
wandb.init(project="my-ml-project", entity="my-username")
```

```
# Training logic...
```

```
model = MyModel()
```

```
# Save the trained model locally
```

```
torch.save(model.state_dict(), "model_v1.pth")
```

```
# Log the model artifact in W&B
```

```
artifact = wandb.Artifact('my-model', type='model')
```

```
artifact.add_file('model_v1.pth')
```

```
wandb.log_artifact(artifact)
```

```
# Finish the run
```

```
wandb.finish()
```

2. Versioning with Automatic Increments

W&B automatically handles versioning for artifacts. If you log a new version of the same model:

Save a new version of the model

```
torch.save(model.state_dict(), "model_v2.pth")
```

Log the new version of the model

```
artifact = wandb.Artifact('my-model', type='model')
```

```
artifact.add_file('model_v2.pth')
```

```
wandb.log_artifact(artifact)
```

3. Retrieving Specific Model Versions

You can easily load a specific version of a model artifact, which is useful when reproducing experiments or deploying a particular model version:

Load a specific version of the model (e.g., version 2)

```
artifact = wandb.use_artifact('my-model:v2')
```

```
artifact_dir = artifact.download()
```

Load the model weights from the downloaded artifact

```
model.load_state_dict(torch.load(f"{artifact_dir}/model_v2.pth"))
```

4. Compare Model Versions

W&B provides tools to compare different versions of the same model:

- **Track performance metrics (e.g., accuracy, loss) across different versions.**
- **Visualize the model's predictions side by side.**
- **Use Parallel Coordinates Plot to compare hyperparameters and performance metrics across multiple versions.**

c. Best Practices for Model Versioning with W&B

- **Consistent Naming:** Use consistent names for your models when logging artifacts. For example, my-model followed by the version number helps in organization.
- **Track Hyperparameters:** Log hyperparameters used in each version for better comparisons.

- **Version Control:** Always log models after significant updates, changes in architecture, or hyperparameters.
-

2. Data Versioning with W&B

a. Why Data Versioning is Important

Data versioning is essential because changes in your dataset can lead to different model performance. It helps in:

- **Reproducibility:** Ensuring the exact same dataset is used when reproducing or debugging an experiment.
- **Collaboration:** Sharing specific versions of the dataset with team members.
- **Tracking Changes:** Documenting changes to the dataset, such as adding new data, cleaning, or transforming the data.

W&B helps manage datasets as artifacts and version them automatically, just like models.

b. Steps to Implement Data Versioning with W&B

1. Logging a Dataset as an Artifact

You can log your dataset as an artifact in W&B:

```
import wandb
```

```
# Initialize a W&B run
```

```
wandb.init(project="my-ml-project", entity="my-username")
```

```
# Add a dataset as an artifact
```

```
artifact = wandb.Artifact('my-dataset', type='dataset')
```

```
artifact.add_file('data.csv') # Path to your dataset
```

```
wandb.log_artifact(artifact)
```

Finish the run

wandb.finish()

2. Automatic Dataset Versioning

If you modify or update your dataset and log it again, W&B will version it automatically:

```
python Copy code  
  
# Log the updated version of the dataset  
artifact = wandb.Artifact('my-dataset', type='dataset')  
artifact.add_file('data_v2.csv')  
wandb.log_artifact(artifact)
```

This will create a new version (`my-dataset:v2`) of the dataset artifact.

3. Using Specific Versions of Datasets

To ensure reproducibility, you can load and use specific versions of datasets:

```
python Copy code  
  
# Load version 2 of the dataset  
artifact = wandb.use_artifact('my-dataset:v2')  
artifact_dir = artifact.download()  
  
# Use the dataset  
data_path = f"{artifact_dir}/data_v2.csv" 
```

4. Data Preprocessing and Versioning

When working with data pipelines, you can also version datasets after transformations, cleaning, or preprocessing:

```
python Copy code  
  
# Save the preprocessed dataset  
preprocessed_data.to_csv("preprocessed_data.csv")  
  
# Log the preprocessed dataset as a new artifact  
artifact = wandb.Artifact('preprocessed-dataset', type='dataset')  
artifact.add_file('preprocessed_data.csv')  
wandb.log_artifact(artifact)
```

This ensures that both raw and processed datasets are versioned and stored.

c. Tracking Dataset Changes

W&B provides the ability to track changes between dataset versions:

- Compare datasets to see added or removed data points.
- Track changes in data distribution, size, or features.
- Visualize differences using W&B's tools to ensure dataset quality.

d. Sharing and Collaboration

- Share Datasets: Team members can access specific versions of the dataset to ensure consistency across different environments.
- Version Control for Datasets: Version control for datasets ensures the correct dataset is always used, especially when datasets are evolving or being frequently updated.

e. Best Practices for Data Versioning with W&B

- Version Raw and Processed Data: Always version both the raw and preprocessed datasets.
- Consistent Data Storage: Use clear and consistent naming conventions for dataset artifacts (e.g., my-dataset:v1, preprocessed-dataset:v2).
- Track Metadata: Log any transformations or changes to the dataset as metadata along with the artifact.

3. Combining Model and Data Versioning

By combining model and data versioning, you can create a robust and reproducible workflow:

- Link Datasets to Models: When logging models, specify which version of the dataset was used for training:

```
model_artifact = wandb.Artifact('my-model', type='model')
```

```
model_artifact.add_file('model.pth')
```

```
# Reference the dataset used to train this model
```

```
model_artifact.add_reference('wandb-artifact://my-dataset:v2')
```

wandb.log_artifact(model_artifact)

- **Reproducibility:** Using dataset and model artifacts ensures that you can fully reproduce an experiment by using the same dataset and model versions.
- **End-to-End Tracking:** You can track the end-to-end workflow, from raw dataset version to model version, using W&B's artifact management.